

Natural Language Processing 2

Building an LLM from Scratch —Data Preparation

Fatemeh Dehrouyeh

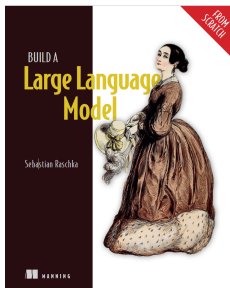


Contents

- 1 Tokenizing text
- 2 Converting into Token IDs
- 3 Adding Special Tokens
- 4 Byte Pair Encoding
- 5 Data sampling with a sliding window
- 6 Creating token embeddings
- 7 Encoding Word Positions

Main Source

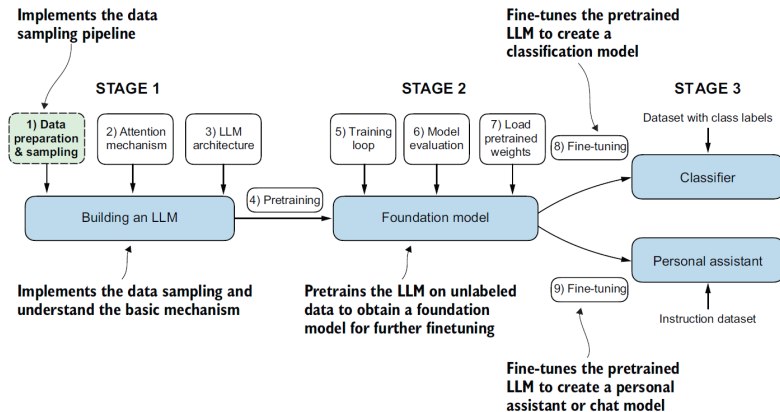
- **Video lectures:** YouTube lecture series
- **Source book:** *Build a Large Language Model (From Scratch)* by Sebastian Raschka
- **GitHub code:** <https://github.com/rasbt/LLMs-from-scratch>



Current Section

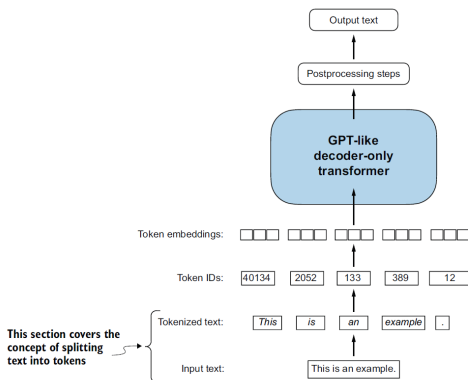
- 1 Tokenizing text
- 2 Converting into Token IDs
- 3 Adding Special Tokens
- 4 Byte Pair Encoding
- 5 Data sampling with a sliding window
- 6 Creating token embeddings
- 7 Encoding Word Positions

Roadmap



Tokenizing text

We split an input text into individual tokens, which are either words or special characters, such as punctuation characters.



Dataset: The Verdict

<https://www.kaggle.com/datasets/golammostofas/the-verdict>

The Verdict

Edith Wharton



1908

Tokenization

```
import os
import urllib.request

if not os.path.exists("the-verdict.txt"):
    url = ("https://raw.githubusercontent.com/rasbt/"
          "LLMs-from-scratch/main/ch02/01_main-chapter-code/"
          "the-verdict.txt")
    file_path = "the-verdict.txt"
    urllib.request.urlretrieve(url, file_path)
```

```
with open("the-verdict.txt", "r", encoding="utf-8") as f:
    raw_text = f.read()

print("Total number of character:", len(raw_text))
print(raw_text[:99])
```

Total number of character: 20479

I HAD always thought Jack Gisburn rather a cheap genius--though a good fellow enough--so it was no



```
import re
```

```
preprocessed = re.split(r'([.,:;?_!"()\'"]|--|\s)', raw_text)
preprocessed = [item.strip() for item in preprocessed if item.strip()]
print(preprocessed[:30])
```

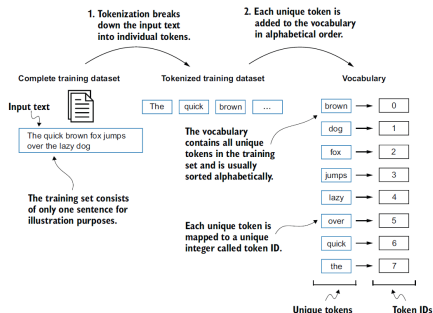
```
... ['I', 'HAD', 'always', 'thought', 'Jack', 'Gisburn', 'rather', 'a', 'cheap', 'genius', '--', 'though',
```


Current Section

- 1 Tokenizing text
- 2 Converting into Token IDs**
- 3 Adding Special Tokens
- 4 Byte Pair Encoding
- 5 Data sampling with a sliding window
- 6 Creating token embeddings
- 7 Encoding Word Positions

Converting into Token IDs

We build a vocabulary by tokenizing the entire text in a training dataset into individual tokens. These individual tokens are then sorted alphabetically, and duplicate tokens are removed. The unique tokens are then aggregated into a vocabulary that defines a mapping from each unique token to a unique integer value. The depicted vocabulary is purposely small and contains no punctuation or special characters for simplicity.



Code

```
all_words = sorted(set(preprocessed))
vocab_size = len(all_words)

print(vocab_size)
```

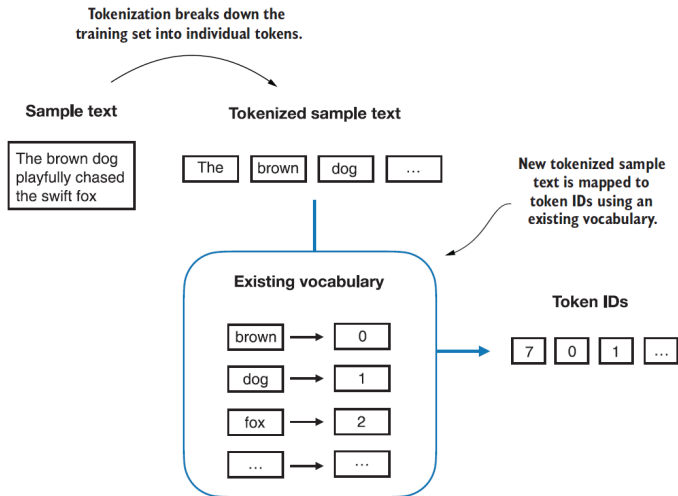
```
1130
```

```
vocab = {token:integer for integer,token in enumerate(all_words)}
```

```
▶ for i, item in enumerate(vocab.items()):
    print(item)
    if i >= 50:
        break
```

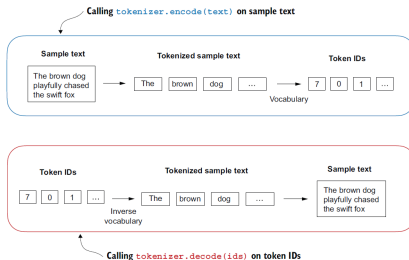
```
... ('!', 0)
    ('"', 1)
    ('"', 2)
    ('(', 3)
    (')', 4)
    (',', 5)
    ('--', 6)
```

Tokenization of a short sample text



Tokenizer Method

Tokenizer implementations share two common methods: an encode method and a decode method. The encode method takes in the sample text, splits it into individual tokens, and converts the tokens into token IDs via the vocabulary. The decode method takes in token IDs, converts them back into text tokens, and concatenates the text tokens into natural text.



Code

```

▶ # The encode function turns text into token IDs
# The decode function turns token IDs back into text

class SimpleTokenizerV1:
    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = {i:s for s,i in vocab.items()}

    def encode(self, text):
        preprocessed = re.split(r'([.,:;?_!"()\'|]|--|\s)', text)

        preprocessed = [
            item.strip() for item in preprocessed if item.strip()
        ]
        ids = [self.str_to_int[s] for s in preprocessed]
        return ids

    def decode(self, ids):
        text = " ".join([self.int_to_str[i] for i in ids])
        # Replace spaces before the specified punctuations
        text = re.sub(r'\s+([.,:;?_!"()\'|])', r'\1', text)
        return text

```

Code Example

```
tokenizer = SimpleTokenizerV1(vocab)

text = """It's the last he painted, you know,"
      Mrs. Gisburn said with pardonable pride."""
ids = tokenizer.encode(text)
print(ids)
```

```
[1, 56, 2, 850, 988, 602, 533, 746, 5, 1126, 596, 5, 1, 67, 7, 38, 851, 1108, 754, 793, 7]
```



```
tokenizer.decode(ids)
```

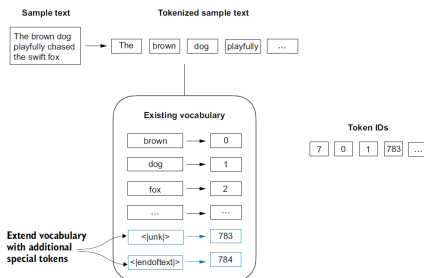
```
... "' It\' s the last he painted, you know," Mrs. Gisburn said with pardonable pride.'
```

Current Section

- 1 Tokenizing text
- 2 Converting into Token IDs
- 3 Adding Special Tokens**
- 4 Byte Pair Encoding
- 5 Data sampling with a sliding window
- 6 Creating token embeddings
- 7 Encoding Word Positions

Adding Special Tokens

We add special tokens to a vocabulary to deal with certain contexts. For instance, we add an `<|unk|>` token to represent new and unknown words that were not part of the training data and thus not part of the existing vocabulary. Furthermore, we add an `<|endoftext|>` token that we can use to separate two unrelated text sources.



Example: The word "Hello" does not exist in the Verdict story

```

tokenizer = SimpleTokenizerV1(vocab)

text = "Hello, do you like tea. Is this-- a test?"

tokenizer.encode(text)

```

```

...
-----
KeyError                                Traceback (most recent call last)
~/tmp/ipykernel_8723/2162118319.py in <cell line: 0>()
      3 text = "Hello, do you like tea. Is this-- a test?"
      4
----> 5 tokenizer.encode(text)

~/tmp/ipykernel_8723/101352538.py in encode(self, text)
     13         item.strip() for item in preprocessed if item.strip()
     14     ]
----> 15     ids = [self.str_to_int[s] for s in preprocessed]
     16     return ids
     17

KeyError: 'Hello'

```

Next steps: [Explain error](#)

The above code produces an error because the word "Hello" is not contained in the vocabulary. To deal with such cases, we can add special tokens like "<|unk|>" to the vocabulary to represent unknown words.

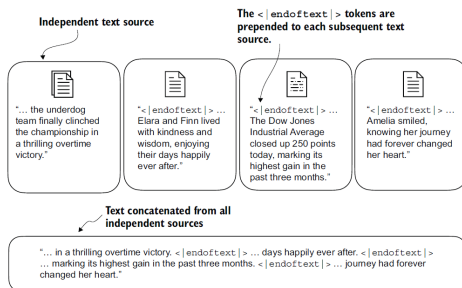
Later, we will see that instead of using an UNK token, we can break a word into subwords or characters so the model can better understand it. This is called Byte Pair Encoding, or BPE.

Other Special Tokens

- Some tokenizers use special tokens to help the LLM with additional context
- Some of these special tokens are
 - `[BOS]` (beginning of sequence) marks the beginning of text
 - `[EOS]` (end of sequence) marks where the text ends (this is usually used to concatenate multiple unrelated texts, e.g., two different Wikipedia articles or two different books, and so on)
 - `[PAD]` (padding) if we train LLMs with a batch size greater than 1 (we may include multiple texts with different lengths; with the padding token we pad the shorter texts to the longest length so that all texts have an equal length)
- `[UNK]` to represent words that are not included in the vocabulary
- Note that GPT-2 does not need any of these tokens mentioned above but only uses an `<|endoftext|>` token to reduce complexity
- The `<|endoftext|>` is analogous to the `[EOS]` token mentioned above
- GPT also uses the `<|endoftext|>` for padding (since we typically use a mask when training on batched inputs, we would not attend padded tokens anyways, so it does not matter what these tokens are)
- GPT-2 does not use an `<UNK>` token for out-of-vocabulary words; instead, GPT-2 uses a byte-pair encoding (BPE) tokenizer, which breaks down words into subword units which we will discuss in a later section

We use the `<|endoftext|>` tokens between two independent sources of text

We use the `<|endoftext|>` tokens between two independent sources of text:



We add another token called "`<|endoftext|>`" which is used in GPT-2 training to denote the end of a text (and it's also used between concatenated text, like if our training datasets consists of multiple articles, books, etc.)

Code

```
all_tokens = sorted(list(set(preprocessed)))
all_tokens.extend(["<|endoftext|>", "<|unk|>"])

vocab = {token:integer for integer,token in enumerate(all_tokens)}
```

```
len(vocab.items())
```

```
1132
```

```
for i, item in enumerate(list(vocab.items())[-5:]):
    print(item)
```

```
('younger', 1127)
('your', 1128)
('yourself', 1129)
('<|endoftext|>', 1130)
('<|unk|>', 1131)
```

Updating SimpleTokenizer Class

- We also need to adjust the tokenizer accordingly so that it knows when and how to use the new `<unk>` token

```

class SimpleTokenizerV2:
    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = { i:s for s,i in vocab.items()}

    def encode(self, text):
        preprocessed = re.split(r'([.,:;?!"()\' ]|--|\s)', text)
        preprocessed = [item.strip() for item in preprocessed if item.strip()]
        preprocessed = [
            item if item in self.str_to_int
            else "<unk>" for item in preprocessed
        ]

        ids = [self.str_to_int[s] for s in preprocessed]
        return ids

    def decode(self, ids):
        text = " ".join([self.int_to_str[i] for i in ids])
        # Replace spaces before the specified punctuations
        text = re.sub(r'\s+([.,:;?!"()\' ])', r'\1', text)
        return text

```

Now there is no error for "Hello"

```
▶ tokenizer = SimpleTokenizerV2(vocab)
  text = "Hello, do you like tea. Is this-- a test?"

  tokenizer.decode(tokenizer.encode(text))

... '<|unk|>, do you like tea. <|unk|> this -- a <|unk|>?'
```

Current Section

- 1 Tokenizing text
- 2 Converting into Token IDs
- 3 Adding Special Tokens
- 4 Byte Pair Encoding**
- 5 Data sampling with a sliding window
- 6 Creating token embeddings
- 7 Encoding Word Positions

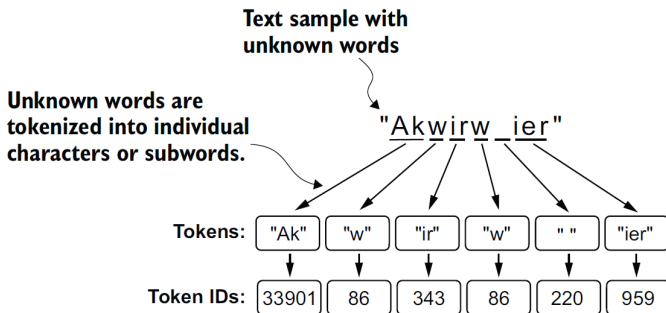
Byte Pair Encoding

GPT-2 used BytePair encoding (BPE) as its tokenizer. It allows the model to break down words that aren't in its predefined vocabulary into smaller subword units or even individual characters, enabling it to handle out-of-vocabulary words.

For instance, if GPT-2's vocabulary doesn't have the word "unfamiliarword," it might tokenize it as ["unfam", "iliar", "word"] or some other subword breakdown, depending on its trained BPE merges.

Example

BPE tokenizers break down unknown words into subwords and individual characters:



Code

```
# !pip install tiktoken

import importlib
import tiktoken

tokenizer = tiktoken.get_encoding("gpt2")

text = (
    "Hello, do you like tea? <|endoftext|> In the sunlit terraces"
    "of someunknownPlace."
)

integers = tokenizer.encode(text, allowed_special={"<|endoftext|>"})

print(integers)

[15496, 11, 466, 345, 588, 8887, 30, 220, 50256, 554, 262, 4252, 18250, 8812, 2114, 1659, 617, 34680, 27271, 13]

strings = tokenizer.decode(integers)

print(strings)

... Hello, do you like tea? <|endoftext|> In the sunlit terracesof someunknownPlace.
```

Note: `<|endoftext|>` is not ordinary text. It is a reserved special token. When the tokenizer sees it in the text, it has two possible ways to handle it: treat it as one special token, or treat it as normal text and split it into smaller tokens. We add `allowed_special="<|endoftext|>"` to explicitly tell the tokenizer to treat `<|endoftext|>` as one valid special token instead of rejecting it.

Try vercel

<https://tiktokenizer.vercel.app/?model=gpt2>

Tiktokenizer

```
howtohandleunknowntokens?
```

gpt2

Token count

11

```
howtohandleunknowntokens?
```

4919, 83, 1219, 392, 293, 2954, 2197, 429, 482, 641, 30

In more details...

BPE starts with letters, then keeps merging the most common neighboring pieces until it builds a useful vocabulary of word parts.

The Main Idea of Byte-Pair-Encoding

Byte Pair Encoding, BPE, turns text into tokens.

Instead of treating every word as one token, BPE learns common pieces of words.

lowest $\rightarrow$ low + est

newer $\rightarrow$ new + er

This helps because models cannot store every possible word.

For example, suppose the tokenizer vocabulary contains:

low, lower, lowest

Then the model sees:

lowermost

A word-level tokenizer may fail. But BPE can split it:

lowermost $\rightarrow$ low + er + most

How BPE learns tokens

BPE starts with characters.

Training words:

low, lower, lowest

Initial split:

low $\rightarrow$ l o w

lower $\rightarrow$ l o w e r

lowest $\rightarrow$ l o w e s t

Then BPE counts neighboring pairs:

$(l, o) : 3, (o, w) : 3, (w, e) : 2$

$(e, r) : 1, (e, s) : 1, (s, t) : 1$

It merges the most common pair (At each merge step, BPE chooses one most frequent pair; if pairs are tied, it uses a fixed tie-breaking rule, such as choosing the pair that appears first in the training data):

$(l, o) \rightarrow lo$

Now lo is a token. BPE repeats the same process again.

The repeated merge process

After the first merge, BPE counts pairs again.

Current words:

lo w, lo w e r, lo w e s t

The pair (*lo, w*) appears 3 times.

So BPE merges:

$(lo, w) \rightarrow low$

Now:

lo w \rightarrow low

lo w e r \rightarrow low e r

lo w e s t \rightarrow low e s t

Then it may merge:

$(low, e) \rightarrow lowe$

$(lowe, r) \rightarrow lower$

The tokenizer is slowly building bigger useful pieces.

The learned vocabulary

After many merges, BPE may know tokens like:

l, o, w, e, r, s, t

lo, low, lowe, lower, est

This vocabulary lets it handle both common and rare words.

Examples:

lower → *lower*

lowest → *low* + *est*

lowly → *low* + *ly*

Common words may become one token. Rare words get split into smaller tokens. Unknown words are still manageable because BPE can fall back to smaller pieces.

Why LLMs use BPE-like tokenization

Large language models usually read tokens, not raw characters. BPE helps because:

- It keeps the vocabulary size manageable.
- It handles rare and unseen words.
- It reuses common word parts.
- It is more efficient than character-by-character reading.

Example:

unbelievable $\rightarrow$ *un* + *believe* + *able*

Core rule:

Repeatedly merge the most common neighboring tokens.

Full process:

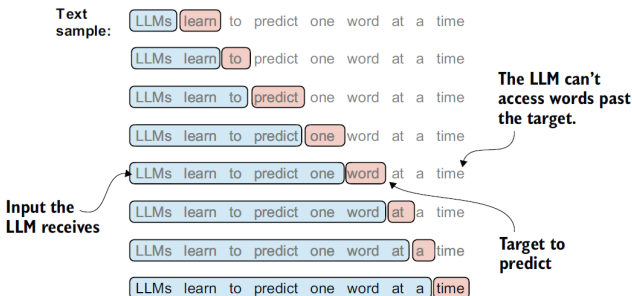
- 1 Start with characters.
- 2 Count neighboring token pairs.
- 3 Merge the most frequent pair.
- 4 Repeat many times.
- 5 Use the learned merges to tokenize new text.

Current Section

- 1 Tokenizing text
- 2 Converting into Token IDs
- 3 Adding Special Tokens
- 4 Byte Pair Encoding
- 5 Data sampling with a sliding window**
- 6 Creating token embeddings
- 7 Encoding Word Positions

Data sampling with a sliding window

We train LLMs to generate one word at a time, so we want to prepare the training data accordingly where the next word in a sequence represents the target to predict



Understanding Concept

For each text chunk, we want the inputs and targets. Since we want the model to predict the next word, the targets are the inputs shifted by one position to the right.

```
with open("the-verdict.txt", "r", encoding="utf-8") as f:  
    raw_text = f.read()
```

```
enc_text = tokenizer.encode(raw_text)  
print(len(enc_text))
```

```
5145
```

```
enc_sample = enc_text[50:]
```

```
▶ context_size = 4
```

```
x = enc_sample[:context_size]  
y = enc_sample[1:context_size+1]
```

```
print(f"x: {x}")  
print(f"y: {y}")
```

```
... x: [290, 4920, 2241, 287]  
    y: [4920, 2241, 287, 257]
```

More Specifically...

One by one, the prediction would look like as follows:

```
for i in range(1, context_size+1):
    context = enc_sample[:i]
    desired = enc_sample[i]

    print(context, "---->", desired)
```

```
[290] ----> 4920
[290, 4920] ----> 2241
[290, 4920, 2241] ----> 287
[290, 4920, 2241, 287] ----> 257
```

```
for i in range(1, context_size+1):
    context = enc_sample[:i]
    desired = enc_sample[i]

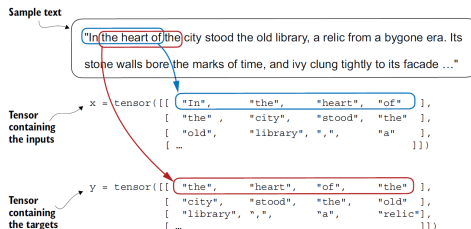
    print(tokenizer.decode(context), "---->", tokenizer.decode([desired]))
```

```
and ----> established
and established ----> himself
and established himself ----> in
and established himself in ----> a
```

We will take care of the next-word prediction in a later chapter after we covered the attention mechanism. For now, we implement a simple data loader that iterates over the input dataset and returns the inputs and targets shifted by one.

Sliding Window

We use a sliding window approach, changing the position by +1:



To implement efficient data loaders, we collect the inputs in a tensor, x , where each row represents one input context. A second tensor, y , contains the corresponding prediction targets (next words), which are created by shifting the input by one position.

Create dataset and dataloader that extract chunks from the input text dataset

```
import torch
from torch.utils.data import Dataset, DataLoader

class GPTDatasetV1(Dataset):
    def __init__(self, txt, tokenizer, max_length, stride):
        self.input_ids = []
        self.target_ids = []

        # Tokenize the entire text
        token_ids = tokenizer.encode(txt, allowed_special={"<|endoftext|>"})
        assert len(token_ids) > max_length, "Number of tokenized inputs must at least be equal to max_length+1"

        # Use a sliding window to chunk the book into overlapping sequences of max_length
        for i in range(0, len(token_ids) - max_length, stride):
            input_chunk = token_ids[i:i + max_length]
            target_chunk = token_ids[i + 1: i + max_length + 1]
            self.input_ids.append(torch.tensor(input_chunk))
            self.target_ids.append(torch.tensor(target_chunk))

    def __len__(self):
        return len(self.input_ids)

    def __getitem__(self, idx):
        return self.input_ids[idx], self.target_ids[idx]
```


Creating DataLoader

```
def create_dataloader_v1(txt, batch_size=4, max_length=256,
                        stride=128, shuffle=True, drop_last=True,
                        num_workers=0):

    # Initialize the tokenizer
    tokenizer = tiktoken.get_encoding("gpt2")

    # Create dataset
    dataset = GPTDatasetV1(txt, tokenizer, max_length, stride)

    # Create dataloader
    dataloader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=shuffle,
        drop_last=drop_last,
        num_workers=num_workers
    )

    return dataloader
```

```
with open("the-verdict.txt", "r", encoding="utf-8") as f:
    raw_text = f.read()

dataloader = create_dataloader_v1(
    raw_text, batch_size=1, max_length=4, stride=1, shuffle=False
)

data_iter = iter(dataloader)
first_batch = next(data_iter)
print(first_batch)

[tensor([[ 40, 367, 2885, 1464]]), tensor([[ 367, 2885, 1464, 1807]])]

▶ second_batch = next(data_iter)
print(second_batch)

... [tensor([[ 367, 2885, 1464, 1807]]), tensor([[2885, 1464, 1807, 3619]])]
```

`drop_last=True` drops the final incomplete batch so every batch has the same size.

Recusing Overlap and Increasing Batch Size

You can see that there is a large overlap between samples in the previous slide. If we choose the stride to be equal to the sequence length, there is no overlap.

```
dataloader = create_dataloader_v1(raw_text, batch_size=8, max_length=4, stride=4, shuffle=False)

data_iter = iter(dataloader)
inputs, targets = next(data_iter)
print("Inputs:\n", inputs)
print("\nTargets:\n", targets)
```

Inputs:

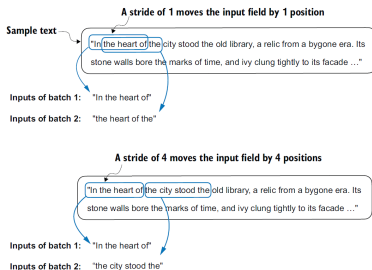
```
tensor([[ 40,   367,  2885,  1464],
        [ 1807,  3619,   402,   271],
        [10899,  2138,   257,  7026],
        [15632,   438,  2016,   257],
        [  922,  5891,  1576,   438],
        [  568,   340,   373,   645],
        [ 1049,  5975,   284,   502],
        [  284,  3285,   326,    11]])
```

Targets:

```
tensor([[ 367,  2885,  1464,  1807],
        [ 3619,   402,   271, 10899],
        [ 2138,   257,  7026, 15632],
        [  438,  2016,   257,   922],
        [ 5891,  1576,   438,   568],
        [  340,   373,   645,  1049],
        [ 5975,   284,   502,   284],
        [ 3285,   326,    11,   287]])
```

Sliding Window Size Effect

When creating multiple batches from the input dataset, we slide an input window across the text. If the stride is set to 1, we shift the input window by one position when creating the next batch. If we set the stride equal to the input window size, we can prevent overlaps between the batches. An example using stride equal to the context length (here: 4) as shown below:



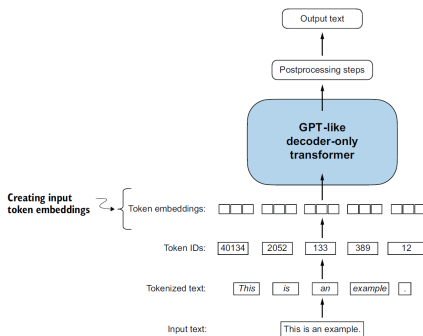
Note that we increase the stride here so that we don't have overlaps between the batches, since more overlap could lead to increased overfitting.

Current Section

- 1 Tokenizing text
- 2 Converting into Token IDs
- 3 Adding Special Tokens
- 4 Byte Pair Encoding
- 5 Data sampling with a sliding window
- 6 Creating token embeddings**
- 7 Encoding Word Positions

Creating token embeddings

The data is already almost ready for an LLM. But lastly let us embed the tokens in a continuous vector representation using an embedding layer. Usually, these embedding layers are part of the LLM itself and are updated (trained) during model training.



Code

```

▶ # Suppose we have the following four input examples with input ids 2, 3, 5, and 1 (after tokenization):
input_ids = torch.tensor([2, 3, 5, 1])

# For the sake of simplicity, suppose we have a small vocabulary of only 6 words and we want to create embeddings of size 3:
vocab_size = 6
output_dim = 3

torch.manual_seed(123)
embedding_layer = torch.nn.Embedding(vocab_size, output_dim)

# This would result in a 6x3 weight matrix:
print(embedding_layer.weight)
# These embedding weights will be optimized during training

... Parameter containing:
tensor([[ 0.3374, -0.1778, -0.1690],
        [ 0.9178,  1.5810,  1.3010],
        [ 1.2753, -0.2010, -0.1606],
        [-0.4015,  0.9666, -1.1481],
        [-1.1589,  0.3255, -0.6315],
        [-2.8400, -0.7849, -1.4096]], requires_grad=True)

```

Converting the input into embedded weights

It is like looking up an index in a table and retrieving the corresponding embedding vector.

```
# To get the embedding weight of the ID 4(index start at 0):
print(embedding_layer(torch.tensor([3])))
```

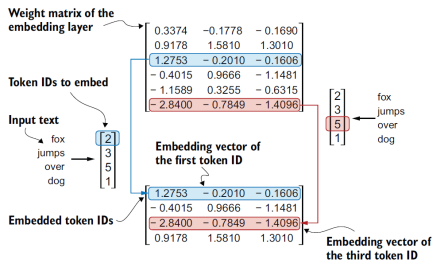
```
tensor([[ -0.4015,  0.9666, -1.1481]], grad_fn=<EmbeddingBackward0>)
```

```
▶ # To embed all four input_ids values
print(embedding_layer(input_ids))
```

```
... tensor([[ 1.2753, -0.2010, -0.1606],
           [-0.4015,  0.9666, -1.1481],
           [-2.8400, -0.7849, -1.4096],
           [ 0.9178,  1.5810,  1.3010]], grad_fn=<EmbeddingBackward0>)
```

Visual Example

Embedding layers perform a lookup operation, retrieving the embedding vector corresponding to the token ID from the embedding layer's weight matrix. For instance, the embedding vector of the token ID 5 is the sixth row of the embedding layer weight matrix (it is the sixth instead of the fifth row because Python starts counting at 0). We assume that the token IDs were produced by the small vocabulary from previous section.



Current Section

- 1 Tokenizing text
- 2 Converting into Token IDs
- 3 Adding Special Tokens
- 4 Byte Pair Encoding
- 5 Data sampling with a sliding window
- 6 Creating token embeddings
- 7 Encoding Word Positions**

Weight matrix of the embedding layer

Token IDs to embed

fox
jumps
over
fox

The same token IDs result in the same embedding vectors

0.3374	-0.1778	-0.1690
0.9178	1.5810	1.3010
1.2753	-0.2010	-0.1606
-0.4015	0.9666	-1.1481
-1.1589	0.3255	-0.6315
-2.8400	-0.7849	-1.4096

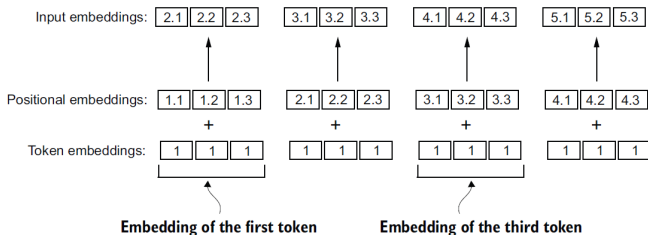
1.2753	- 0.2010	- 0.1606
- 0.4015	0.9666	- 1.1481
- 2.8400	- 0.7849	- 1.4096
1.2753	- 0.2010	- 0.1606

$$\begin{bmatrix} 2 \\ 3 \\ 5 \\ 2 \end{bmatrix}$$

fox
_ jumps
over
fox

Positional Embeddings

Positional embeddings are combined with the token embedding vector to form the input embeddings for a large language model:



Positional embeddings are added to the token embedding vector to create the input embeddings for an LLM. The positional vectors have the same dimension as the original token embeddings. The token embeddings are shown with value 1 for simplicity.

Code

```

▶ # The BytePair encoder has a vocabulary size of 50,257:
# Suppose we want to encode the input tokens into a 256-dimensional vector representation:

vocab_size = 50257
output_dim = 256

token_embedding_layer = torch.nn.Embedding(vocab_size, output_dim)

# If we sample data from the dataloader, we embed the tokens in each batch into a 256-dimensional vector
# If we have a batch size of 8 with 4 tokens each, this results in a 8 x 4 x 256 tensor:
max_length = 4
dataloader = create_dataloader_v1(
    raw_text, batch_size=8, max_length=max_length,
    stride=max_length, shuffle=False
)
data_iter = iter(dataloader)
inputs, targets = next(data_iter)

print("Token IDs:\n", inputs)
print("\nInputs shape:\n", inputs.shape)

... Token IDs:
  tensor([[ 40,   367,  2885, 1464],
         [1807,  3619,   402,   271],
         [10899,  2138,   257,  7026],
         [15632,   438,  2016,   257],
         [ 922,  5891,  1576,   438],
         [ 568,   340,   373,   645],
         [1049,  5975,   284,   502],
         [ 284,  3285,   326,    11]])

Inputs shape:
  torch.Size([8, 4])

```

Combining Token Embeddings and Positional Embeddings

```

token_embeddings = token_embedding_layer(inputs)
print(token_embeddings.shape)

# GPT-2 uses absolute position embeddings, so we just create another embedding layer:
context_length = max_length
pos_embedding_layer = torch.nn.Embedding(context_length, output_dim)

# torch.arange(max_length) = [0, 1, 2, 3] is a placeholder for positions 0, 1, 2, and 3.
pos_embeddings = pos_embedding_layer(torch.arange(max_length))
print(pos_embeddings.shape)

# To create the input embeddings used in an LLM, we simply add the token and the positional embeddings:
input_embeddings = token_embeddings + pos_embeddings
print(input_embeddings.shape)

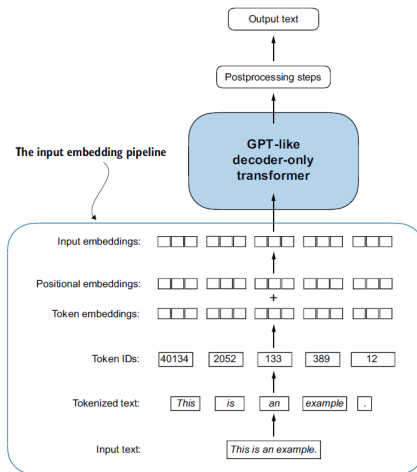
... torch.Size([8, 4, 256])
    torch.Size([4, 256])
    torch.Size([8, 4, 256])

```

This code adds positional embeddings to token embeddings so the model knows both the meaning of each token and its position in the sequence.

Position embeddings are also optimized during training, and all samples at the same position in the batch receive the same position embedding, so we do not need to manually duplicate it across the batch dimension.

Now We Know About Data Preparation Part!



Congratulations!

Now you know how to prepare data for an LLM!

